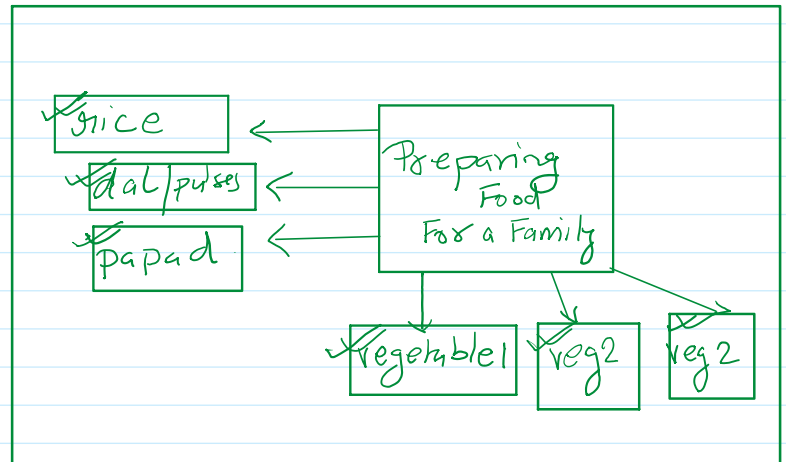


Methods in Java

28 April 2026 06:02

In Java, a method is used to do a specific work and helps make the program simple and reusable.



Food is ready to consume.

What is a method in Java?

A method is a block of code, or a collection of statements, or a set of code grouped to perform a certain task or operation. It is used to achieve the reusability of code. We write a method once and use it many times. We do not need to write code again and again. It also provides easy modification and readability of code just by adding or removing a chunk of code. The method is executed only when we call or invoke it.

The most important method in Java is the `main()` method.

Advantages of Using Methods

The following are the advantages of using methods in Java:

- **Reusability:** Methods provide code reusability. Once a method is defined, it can be called multiple times for various parts of the program.
- **Modularity:** Methods help breakdown complex problem into smaller and manageable pieces.
- **Maintainability:** With methods, updating and debugging code become easier, since changes in one part of the program do not necessarily impact other parts.
- **Abstraction:** Method provides a way to encapsulate implementation details, exposing only the necessary parts to the user.

Method naming rules:

While defining a method, remember that the method name must be a verb and start with a lowercase letter. If the method name has more than two words, the first name must be a verb followed by an adjective or noun. In the multi word method name, the first letter of each word must be in uppercase except the first word. For example:

Single-word method name: `sum()`, `area()`

Multi-word method name: `areaOfCircle()`, `stringComparison()`

It is also possible that a method has same name as another method in the same class. It is known as method overloading.

`substring(3);`

```
substring(3, 7);
```

```
class String{
```

```
    ...  
    public String substring(int start){
```

```
        ...  
    }
```

```
    public String substring(int start, int end){
```

```
        ...  
    }
```

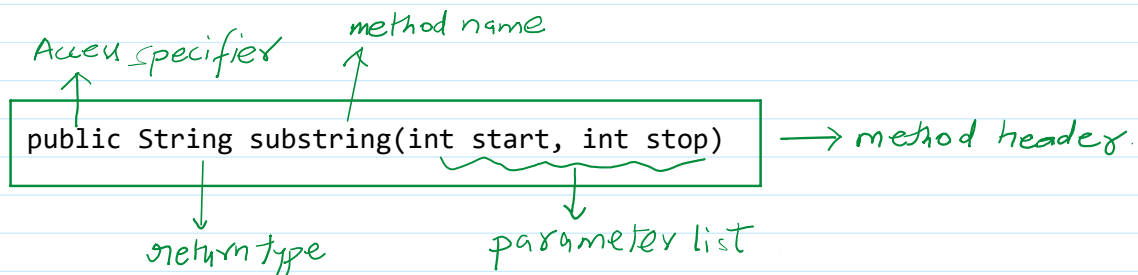
```
    ...
```

```
}
```

object
String str;
str.substring(3);
str.substring(3, 7);

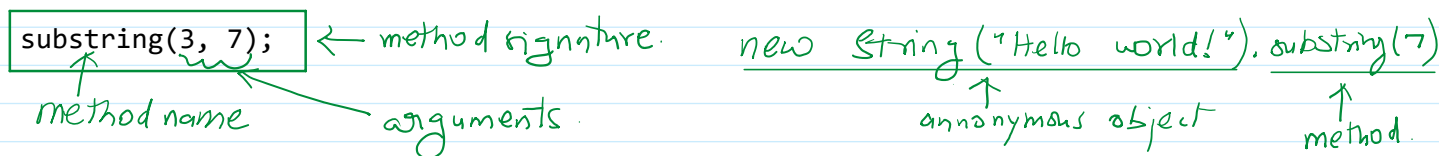
Method Declaration

The method declaration provides information about method attributes. Such as visibility, return type, name and arguments. It has six components that are known as method header.



Method Signature

Every method has a method signature. It is the part of the method declaration. It includes the method name and argument list.



- **Note:** Method signature is used to call the method.

Access Specifier

Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides four types of access specifiers:

- **public:** The method is accessible to all classes when we use the public specifier in our application.
- **private:** When we use private access specifier, the method is accessible only in the classes in which it is defined.
- **protected:** When we use a protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **default:** We do not need to use any access specifier in the method

declaration. Java uses the default access specifier by default. It is visible only from the same package.

Return type: Return type is a data type that the method returns. It may have a primitive data type, object, Collection, void etc. If the method does not return anything, we use the void keyword.

Method Name: It is a unique name that is used to define the name of a method. It must correspond to the functionality of the method. Suppose if we are creating a method for subtraction of two numbers, the method name must be subtraction() A method is invoked by its name.

Parameter list: It is a list of parameters separated by a comma and enclosed in a pair of parentheses. It contains the data type and variable name. If method has no parameter, leave the parenthesis blank.

Method Body: It is a part of a method declaration. It contains all the actions to be performed. It is enclosed within a pair of curly braces.

package Loops;

```
/**
 * A prime number is said to be Twisted Prime", if the new number obtained
 * reversing the digits is also a prime number.
 * Write a program to accept a number and check whether the number
 * is 'Twisted Prime' or not.
 * Sample Input: 167
 * Sample Output: 761
 * 167 is a Twisted Prime.
 * @author (your name)
 * @version (a version number or a date)
 */
```

```
public class Q12
{
    public boolean isPrime(int x){
        if(x > 2 && (x % 2 == 0) || x == 1){
            return false;
        }
        int d;
        for(d = 3; d <= Math.sqrt(x); d += 2){
            if (x % d == 0){
                return false;
            }
        }
        return true;
    }

    public int reverse(int x){
        int n = 0;
        for(;x>0; n = n * 10 + x % 10, x /=10);
        return n;
    }

    public boolean isTwistedPrime(int x){
```

class.
static-method
↑
needs class reference
to call.
OR
the static present in same
class can be invoked or called
directly.

```

        return isPrime(x) && isPrime(reverse(x));
    }

    public static void main(String[] args){
        System.out.print("\nEnter any integer value: ");
        int n = new java.util.Scanner(System.in).nextInt();
        Q12 ob = new Q12();
        if(ob.isTwistedPrime(n)){
            System.out.print("\n" + n + " is twisted prime.");
        }
        else{
            System.out.print("\n" + n + " is not twisted prime.");
        }
    }
}

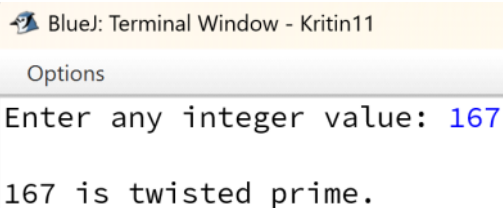
```

Output:

Example of non-static method.

Create object to call non-static method.

needs object of that class as reference to call the method.



```

Blue: Terminal Window - Kritin11
Options
Enter any integer value: 167
167 is twisted prime.

```

Types of Method

There are two types of method in Java:

- **Predefined Method:** The methods that are already defined in Java class library are known as predefined methods. It is also known as the standard library method or built in method. We can directly use these methods just by calling them in the program at any point. For example: `Math.sqrt(x)`;
- **User-defined Method:** The method written by user or programmer is known as a user defined method. These methods can be modified or customized according to the requirements.

Calling User defined method

In Java, calling or invoking a method depends on whether the method is static or non-static.

If the method is static, we can call it directly using the class name or from within the same class. Note that we need to create an object for calling non-static method.

Rule:

- If the method is non static, we must create an object of the class and use it to call the method.
- Will get compile time error if we try to call a non-static method from a static context without an object. Because non static methods belongs to instances, not the class itself.

